8. HotStuff 성능 최적화

8.1 성능 최적화 개요

8.1.1 성능 지표

HotStuff에서 중요한 성능 지표들:

- **처리량 (Throughput)**: 초당 처리되는 트랜잭션 수
- **지연 시간 (Latency)**: 트랜잭션 제출부터 커밋까지의 시간
- **확장성 (Scalability)**: 노드 수 증가에 따른 성능 변화
- **자원 사용량**: CPU, 메모리, 네트워크 대역폭 사용량

8.1.2 최적화 영역

```
┌─────────────────┐ ┌─────────────────┐ ┌─────────────────┐

│   암호화 최적화  │ │  네트워크 최적화  │ │   메모리 최적화   │

│                                   │ │                                 │ │

│ • 서명 집계      │ │ • 배치 전송      │ │ • 객체 풀링      │

│ • 병렬 검증      │ │ • 압축           │ │ • 캐싱           │

│ • 하드웨어 가속   │ │ • 연결 재사용    │ │ • GC 최적화      │

└─────────────────┘ └─────────────────┘ └─────────────────┘

          │                                         │

      └──────────────────────┼──────────────────────┘
```

```
                    |
┌──────────────────┐
|  알고리즘 최적화  |
|                  |
| • 파이프라이닝    |
| • 조기 종료      |
| • 적응적 타임아웃 |
└──────────────────┘
```

8.2 암호화 최적화

8.2.1 서명 집계 최적화

```
// BLS 서명의 효율적 집계
type optimizedBLSCrypto struct {
*blsCrypto
// 서명 캐시
signatureCache  map[string]bls.Signature
cacheMutex sync.RWMutex
// 배치 처리
batchSize  int
batchTimeout time.Duration
pendingBatch []signatureRequest
```

```
batchMutex sync.Mutex
}

type signatureRequest struct {
data []byte
response  chan signatureResponse
}

type signatureResponse struct {
signature bls.Signature
error  error
}
```

8.2.2 배치 서명 생성

```
func (obc *optimizedBLSCrypto) Sign(data []byte) (hotstuff.QuorumSignature, erro
// 캐시 확인
key := string(data)
obc.cacheMutex.RLock()
if  cached, exists := obc.signatureCache[key];  exists {
obc.cacheMutex.RUnlock()
return blsSignature{sig: cached, participants: obc.createParticipantSet()}, nil
}
```

```
obc.cacheMutex.RUnlock()

// 배치 처리 큐에 추가

response := make(chan signatureResponse, 1)

request := signatureRequest{data: data, response: response}

obc.batchMutex.Lock()

obc.pendingBatch = append(obc.pendingBatch, request)

// 배치 크기 도달 시 즉시 처리

if  len(obc.pendingBatch) >= obc.batchSize {

obc.processBatch()

}

obc.batchMutex.Unlock()

// 응답 대기

resp := <-response

return blsSignature{sig: resp.signature, participants: obc.createParticipantSet(

}



func (obc *optimizedBLSCrypto) processBatch() {

if  len(obc.pendingBatch) == 0 {

return

}

batch := obc.pendingBatch

obc.pendingBatch = make([]signatureRequest, 0, obc.batchSize)

// 배치 서명 생성 (병렬 처리)
```

```go
go func() {
for _, request := range batch {
sig := obc.privKey.Sign(request.data)
// 캐시에 저장
obc.cacheMutex.Lock()
obc.signatureCache[string(request.data)] = sig
obc.cacheMutex.Unlock()
request.response <- signatureResponse{signature: sig, error: nil}
}
}()
}
```

8.2.3 병렬 서명 검증

```go
func (obc *optimizedBLSCrypto) BatchVerify(signatures []hotstuff.QuorumSignature
results := make([]bool, len(signatures))
// 워커 풀 사용
numWorkers := runtime.NumCPU()
jobs := make(chan verifyJob, len(signatures))
resultsChan := make(chan verifyResult, len(signatures))
// 워커 시작
for i := 0; i < numWorkers; i++ {
go obc.verifyWorker(jobs, resultsChan)
```

```go
}
// 작업 분배
for i, sig := range signatures {
jobs <- verifyJob{index: i, signature: sig, data: data[i]}
}
close(jobs)
// 결과 수집
for i := 0; i < len(signatures); i++ {
result := <-resultsChan
results[result.index] = result.valid
}
return results
}

type verifyJob struct {
index int
signature hotstuff.QuorumSignature
data []byte
}

type verifyResult struct {
index int
valid bool
```

```
}

func (obc *optimizedBLSCrypto) verifyWorker(jobs <-chan verifyJob, results  chan

for  job := range  jobs {

valid := obc.Verify(job.signature, job.data)

results <- verifyResult{index: job.index, valid: valid}

}

}
```

8.2.4 하드웨어 가속 활용

```
// Intel AES-NI 명령어 활용

type hardwareAcceleratedCrypto struct {

*ecdsaCrypto

useAESNI  bool

}

func (hac *hardwareAcceleratedCrypto) Sign(data []byte) (hotstuff.QuorumSignatur

var  hash []byte

if  hac.useAESNI && cpuid.CPU.AES() {

// 하드웨어 가속 해시

hash = hac.hardwareHash(data)
```

```go
} else {
// 소프트웨어 해시
h := sha256.Sum256(data)
hash = h[:]
}
return hac.signHash(hash)
}

func (hac *hardwareAcceleratedCrypto) hardwareHash(data []byte) []byte {
// AES-NI를 활용한 빠른 해시 계산
// 실제 구현은 어셈블리 또는 CGO 사용
return sha256.Sum256(data)[:]
}
```

8.3 네트워크 최적화

8.3.1 메시지 배치 처리

```go
type batchedConfiguration struct {
*Config
// 배치 설정
batchSize int
batchTimeout time.Duration
```

```go
// 배치 큐
proposalBatch []hotstuff.ProposeMsg
voteBatch []hotstuff.VoteMsg
timeoutBatch []hotstuff.TimeoutMsg
// 타이머
batchTimer *time.Timer
batchMutex sync.Mutex
}


func (bc *batchedConfiguration) Propose(proposal hotstuff.ProposeMsg) {
bc.batchMutex.Lock()
defer bc.batchMutex.Unlock()
bc.proposalBatch = append(bc.proposalBatch, proposal)
if len(bc.proposalBatch) >= bc.batchSize {
bc.flushProposals()
} else if len(bc.proposalBatch) == 1 {
// 첫 번째 메시지면 타이머 시작
bc.batchTimer = time.AfterFunc(bc.batchTimeout, func() {
bc.batchMutex.Lock()
bc.flushProposals()
bc.batchMutex.Unlock()
})
}
```

```go
}


func (bc *batchedConfiguration) flushProposals() {
if len(bc.proposalBatch) == 0 {
return
}
if bc.batchTimer != nil {
bc.batchTimer.Stop()
bc.batchTimer = nil
}
// 배치 전송
batch := bc.proposalBatch
bc.proposalBatch = make([]hotstuff.ProposeMsg, 0, bc.batchSize)
go bc.sendProposalBatch(batch)
}
```

8.3.2 메시지 압축

```go
type compressedTransport struct {
transport grpc.ClientConnInterface
// 압축 설정
compressionThreshold int
compressionLevel int
```

```go
}


func (ct *compressedTransport) Invoke(ctx context.Context, method  string, args

// 메시지 크기 확인

data, err := proto.Marshal(args.(proto.Message))

if  err != nil {

return  err

}

if  len(data) > ct.compressionThreshold {

// 압축 적용

compressed, err := ct.compress(data)

if  err == nil && len(compressed) < len(data) {

// 압축된 데이터 사용

opts = append(opts, grpc.UseCompressor("gzip"))

}

}

return  ct.transport.Invoke(ctx, method, args, reply, opts...)

}


func (ct *compressedTransport) compress(data []byte) ([]byte, error) {

var  buf bytes.Buffer

writer, err := gzip.NewWriterLevel(&buf, ct.compressionLevel)

if  err != nil {
```

```go
return nil, err
}
_, err = writer.Write(data)
if err != nil {
return nil, err
}
err = writer.Close()
if err != nil {
return nil, err
}
return buf.Bytes(), nil
}
```

8.3.3 연결 풀링 최적화

```go
type optimizedConnectionPool struct {
pools map[string]*connectionPool
mutex sync.RWMutex
// 풀 설정
maxConnections int
connectionTimeout time.Duration
idleTimeout time.Duration
}
```

```go
type connectionPool struct {
connections  chan *grpc.ClientConn
address  string
dialOpts []grpc.DialOption
// 통계
activeConnections  int32
totalConnections  int32
// 정리
lastUsed time.Time
mutex sync.Mutex
}


func (ocp *optimizedConnectionPool) getConnection(address  string, opts ...grpc.
ocp.mutex.RLock()
pool, exists := ocp.pools[address]
ocp.mutex.RUnlock()
if !exists {
ocp.mutex.Lock()
// 다시 확인 (double-checked locking)
if  pool, exists = ocp.pools[address]; !exists {
pool = &connectionPool{
connections: make(chan *grpc.ClientConn, ocp.maxConnections),
```

```
address: address,
dialOpts: opts,
lastUsed: time.Now(),
}
ocp.pools[address] = pool
}
ocp.mutex.Unlock()
}
return  pool.getConnection()
}


func (cp *connectionPool) getConnection() (*grpc.ClientConn, error) {
cp.mutex.Lock()
cp.lastUsed = time.Now()
cp.mutex.Unlock()
select {
case  conn := <-cp.connections:
// 기존 연결 재사용
atomic.AddInt32(&cp.activeConnections, 1)
return  conn, nil
default:
// 새 연결 생성
if  atomic.LoadInt32(&cp.totalConnections) < int32(len(cp.connections)) {
```

```go
conn, err := grpc.Dial(cp.address, cp.dialOpts...)
if err != nil {
return nil, err
}
atomic.AddInt32(&cp.totalConnections, 1)
atomic.AddInt32(&cp.activeConnections, 1)
return conn, nil
}
// 연결 대기
conn := <-cp.connections
atomic.AddInt32(&cp.activeConnections, 1)
return conn, nil
}
}

func (cp *connectionPool) returnConnection(conn *grpc.ClientConn) {
atomic.AddInt32(&cp.activeConnections, -1)
select {
case cp.connections <- conn:
// 풀에 반환 성공
default:
// 풀이 가득 참, 연결 종료
conn.Close()
```

```
atomic.AddInt32(&cp.totalConnections, -1)
}
}
```

8.4 메모리 최적화

8.4.1 객체 풀링

```
// 블록 객체 풀
var  blockPool = sync.Pool{
New: func() interface{} {
return &hotstuff.Block{}
},
}

// 메시지 객체 풀
var  messagePool = sync.Pool{
New: func() interface{} {
return &hotstuff.ProposeMsg{}
},
}

func  NewBlock(parent hotstuff.Hash, cert hotstuff.QuorumCert, cmd hotstuff.Comm
```

```go
// 풀에서 블록 가져오기
block := blockPool.Get().(*hotstuff.Block)
// 초기화
*block = hotstuff.Block{
parent: parent,
cert: cert,
cmd: cmd,
view: view,
proposer: proposer,
ts: time.Now(),
}
// 해시 계산
block.hash = sha256.Sum256(block.ToBytes())
return  block
}

func  ReleaseBlock(block *hotstuff.Block) {
// 블록 초기화
*block = hotstuff.Block{}
// 풀에 반환
blockPool.Put(block)
}
```

8.4.2 메모리 풀 관리

```
type memoryPool struct {
pools  map[reflect.Type]*sync.Pool
mutex sync.RWMutex
// 통계
allocations  map[reflect.Type]int64
deallocations  map[reflect.Type]int64
statsMutex sync.RWMutex
}

func (mp *memoryPool) Get(objType reflect.Type) interface{} {
mp.mutex.RLock()
pool, exists := mp.pools[objType]
mp.mutex.RUnlock()
if !exists {
mp.mutex.Lock()
if  pool, exists = mp.pools[objType]; !exists {
pool = &sync.Pool{
New: func() interface{} {
return  reflect.New(objType).Interface()
},
}
```

```
mp.pools[objType] = pool
}
mp.mutex.Unlock()
}
obj := pool.Get()
// 통계 업데이트
mp.statsMutex.Lock()
mp.allocations[objType]++
mp.statsMutex.Unlock()
return obj
}

func (mp *memoryPool) Put(obj interface{}) {
objType := reflect.TypeOf(obj).Elem()
mp.mutex.RLock()
pool, exists := mp.pools[objType]
mp.mutex.RUnlock()
if exists {
// 객체 초기화
mp.resetObject(obj)
pool.Put(obj)
// 통계 업데이트
mp.statsMutex.Lock()
```

```
mp.deallocations[objType]++
mp.statsMutex.Unlock()
}
}
```

8.4.3 가비지 컬렉션 최적화

```
type gcOptimizer struct {
// GC 튜닝 파라미터
targetGCPercent  int
maxHeapSize  int64
// 모니터링
lastGCTime time.Time
gcCount  int64
totalGCPause time.Duration
// 적응적 조정
adjustmentInterval time.Duration
lastAdjustment time.Time
}

func (gco *gcOptimizer) Start() {
// 초기 GC 설정
debug.SetGCPercent(gco.targetGCPercent)
```

```go
// 주기적 모니터링
ticker := time.NewTicker(gco.adjustmentInterval)
go func() {
for range ticker.C {
gco.adjustGCSettings()
}
}()
}

func (gco *gcOptimizer) adjustGCSettings() {
var memStats runtime.MemStats
runtime.ReadMemStats(&memStats)
// 힙 사용량 확인
heapUsage := float64(memStats.HeapInuse) / float64(gco.maxHeapSize)
var newGCPercent int
switch {
case heapUsage > 0.8:
// 메모리 사용량이 높으면 더 자주 GC
newGCPercent = gco.targetGCPercent / 2
case heapUsage < 0.3:
// 메모리 사용량이 낮으면 덜 자주 GC
newGCPercent = gco.targetGCPercent * 2
default:
```

```
newGCPercent = gco.targetGCPercent
}
if  newGCPercent != debug.SetGCPercent(-1) {
debug.SetGCPercent(newGCPercent)
}
}
```

8.5 알고리즘 최적화

8.5.1 파이프라인 최적화

```
type pipelinedConsensus struct {
*consensusBase
// 파이프라인 단계
stages []pipelineStage
// 병렬 처리
workers  int
workQueues []chan pipelineWork
}

type pipelineStage interface {
Process(work pipelineWork) pipelineWork
Name() string
```

```go
}

type pipelineWork struct {
proposal hotstuff.ProposeMsg
stage  int
result  chan pipelineResult
}

type pipelineResult struct {
success  bool
error  error
data  interface{}
}

func (pc *pipelinedConsensus) OnPropose(proposal hotstuff.ProposeMsg) {
work := pipelineWork{
proposal: proposal,
stage: 0,
result: make(chan pipelineResult, 1),
}
// 첫 번째 단계 큐에 추가
pc.workQueues[0] <- work
// 결과 대기 (비동기)
```

```go
go func() {
result := <-work.result
if result.success {
// 다음 단계 처리
pc.processNextStage(work)
}
}()
}

func (pc *pipelinedConsensus) processNextStage(work pipelineWork) {
if work.stage >= len(pc.stages) {
// 모든 단계 완료
return
}
// 다음 단계 큐에 추가
work.stage++
if work.stage < len(pc.workQueues) {
pc.workQueues[work.stage] <- work
}
}
```

8.5.2 조기 종료 최적화

```go
type earlyTerminationConsensus struct {
*consensusBase
// 조기 종료 조건
earlyCommitThreshold  int
fastPathEnabled  bool
}


func (etc *earlyTerminationConsensus) OnPropose(proposal hotstuff.ProposeMsg) {
// 기본 검증
if !etc.basicValidation(proposal) {
return
}
// 빠른 경로 시도
if  etc.fastPathEnabled && etc.canUseFastPath(proposal) {
if  etc.tryFastCommit(proposal) {
return  // 빠른 커밋 성공
}
}
// 일반 경로
etc.consensusBase.OnPropose(proposal)
}
```

```go
func (etc *earlyTerminationConsensus) canUseFastPath(proposal hotstuff.ProposeMs

// 빠른 경로 조건 확인

// 1. 네트워크 상태가 좋음

// 2. 최근 타임아웃이 없음

// 3. 충분한 연결된 복제본

return  etc.networkHealthy() &&

etc.noRecentTimeouts() &&

etc.sufficientConnections()

}


func (etc *earlyTerminationConsensus) tryFastCommit(proposal hotstuff.ProposeMsg

// 조기 쿼럼 수집 시도

votes := etc.collectVotesWithTimeout(proposal, time.Millisecond*100)

if  len(votes) >= etc.earlyCommitThreshold {

// 조기 커밋

etc.fastCommit(proposal.Block, votes)

return true

}

return false

}
```

8.5.3 적응적 타임아웃

```go
type adaptiveViewDuration struct {
baseDuration time.Duration
currentDuration time.Duration
// 적응 파라미터
increaseMultiplier  float64
decreaseMultiplier  float64
maxDuration time.Duration
minDuration time.Duration
// 네트워크 상태 추적
recentLatencies []time.Duration
successRate  float64
// 통계
timeoutCount  int64
successCount  int64
totalViews  int64
}

func (avd *adaptiveViewDuration) Duration() time.Duration {
return  avd.currentDuration
}

func (avd *adaptiveViewDuration) ViewTimeout() {
```

```
atomic.AddInt64(&avd.timeoutCount, 1)

atomic.AddInt64(&avd.totalViews, 1)

// 네트워크 상태 분석

avgLatency := avd.calculateAverageLatency()

currentSuccessRate := float64(avd.successCount) / float64(avd.totalViews)

// 적응적 조정

if  currentSuccessRate < 0.8 {

// 성공률이 낮으면 타임아웃 증가

newDuration := time.Duration(float64(avd.currentDuration) * avd.increaseMultipli

if  newDuration <= avd.maxDuration {

avd.currentDuration = newDuration

}

} else  if  avgLatency > 0 {

// 평균 지연시간 기반 조정

targetDuration := avgLatency * 3  // 3배 여유

if  targetDuration > avd.minDuration && targetDuration < avd.maxDuration {

avd.currentDuration = targetDuration

}

}

}



func (avd *adaptiveViewDuration) ViewSucceeded() {

atomic.AddInt64(&avd.successCount, 1)
```

```
atomic.AddInt64(&avd.totalViews, 1)

// 성공 시 타임아웃 감소 고려

currentSuccessRate := float64(avd.successCount) / float64(avd.totalViews)

if  currentSuccessRate > 0.95 {

newDuration := time.Duration(float64(avd.currentDuration) * avd.decreaseMultipli

if  newDuration >= avd.minDuration {

avd.currentDuration = newDuration

}

}

}
```

8.6 캐싱 전략

8.6.1 다층 캐시 시스템

```
type multiLevelCache struct {

// L1: 메모리 캐시 (가장 빠름)

l1Cache  map[string]interface{}

l1Mutex sync.RWMutex

l1Size  int

l1Max  int

// L2: LRU 캐시 (중간 속도)

l2Cache *lru.Cache
```

```
l2Mutex sync.RWMutex
// L3: 디스크 캐시 (가장 느림)
l3Cache diskCache
// 통계
l1Hits, l1Misses int64
l2Hits, l2Misses int64
l3Hits, l3Misses int64
}

func (mlc *multiLevelCache) Get(key string) (interface{}, bool) {
// L1 캐시 확인
mlc.l1Mutex.RLock()
if value, exists := mlc.l1Cache[key]; exists {
mlc.l1Mutex.RUnlock()
atomic.AddInt64(&mlc.l1Hits, 1)
return value, true
}
mlc.l1Mutex.RUnlock()
atomic.AddInt64(&mlc.l1Misses, 1)
// L2 캐시 확인
mlc.l2Mutex.RLock()
if value, exists := mlc.l2Cache.Get(key); exists {
mlc.l2Mutex.RUnlock()
```

```
atomic.AddInt64(&mlc.l2Hits, 1)
// L1으로 승격
mlc.promoteToL1(key, value)
return  value, true
}
mlc.l2Mutex.RUnlock()
atomic.AddInt64(&mlc.l2Misses, 1)
// L3 캐시 확인
if  value, exists := mlc.l3Cache.Get(key);  exists {
atomic.AddInt64(&mlc.l3Hits, 1)
// L2로 승격
mlc.promoteToL2(key, value)
return  value, true
}
atomic.AddInt64(&mlc.l3Misses, 1)
return  nil, false
}
```

8.6.2 블록 캐시 최적화

```
type optimizedBlockChain struct {
*blockChain
// 캐시 계층
```

```
hotCache  map[hotstuff.Hash]*hotstuff.Block // 최근 접근된 블록
warmCache *lru.Cache // LRU 캐시
coldCache diskCache // 디스크 저장
// 캐시 정책
hotCacheSize  int
warmCacheSize  int
// 접근 패턴 추적
accessPattern  map[hotstuff.Hash]accessInfo
patternMutex sync.RWMutex
}


type accessInfo struct {
count  int64
lastAccess time.Time
frequency  float64
}


func (obc *optimizedBlockChain) Get(hash hotstuff.Hash) (*hotstuff.Block, bool)
// 접근 패턴 업데이트
obc.updateAccessPattern(hash)
// 핫 캐시 확인
if  block, exists := obc.hotCache[hash];  exists {
return  block, true
```

```
}
// 웜 캐시 확인
if value, exists := obc.warmCache.Get(hash); exists {
block := value.(*hotstuff.Block)
// 핫 캐시로 승격 고려
if obc.shouldPromoteToHot(hash) {
obc.promoteToHot(hash, block)
}
return block, true
}
// 콜드 캐시 확인
if block, exists := obc.coldCache.Get(hash); exists {
// 웜 캐시로 승격
obc.warmCache.Add(hash, block)
return block, true
}
return nil, false
}

func (obc *optimizedBlockChain) shouldPromoteToHot(hash hotstuff.Hash) bool {
obc.patternMutex.RLock()
info, exists := obc.accessPattern[hash]
obc.patternMutex.RUnlock()
```

```go
if !exists {
return false
}
// 접근 빈도와 최근성 기반 결정
recency := time.Since(info.lastAccess)
return  info.frequency > 0.1 && recency < time.Minute*5
}
```

8.7 성능 모니터링

8.7.1 메트릭 수집

```go
type performanceMonitor struct {
// 처리량 메트릭
transactionCount  int64
blockCount  int64
startTime time.Time
// 지연시간 메트릭
latencyHistogram *prometheus.HistogramVec
// 자원 사용량 메트릭
cpuUsage  float64
memoryUsage  int64
networkIO  int64
```

```go
// 커스텀 메트릭
customMetrics  map[string]interface{}
metricsMutex sync.RWMutex
}


func (pm *performanceMonitor) RecordTransaction(latency time.Duration) {
atomic.AddInt64(&pm.transactionCount, 1)
// 히스토그램에 지연시간 기록
pm.latencyHistogram.WithLabelValues("transaction").Observe(latency.Seconds())
}


func (pm *performanceMonitor) GetThroughput() float64 {
elapsed := time.Since(pm.startTime).Seconds()
return  float64(atomic.LoadInt64(&pm.transactionCount)) / elapsed
}


func (pm *performanceMonitor) GetAverageLatency() time.Duration {
// Prometheus 메트릭에서 평균 계산
metric := &dto.Metric{}
pm.latencyHistogram.WithLabelValues("transaction").Write(metric)
if  metric.Histogram.GetSampleCount() > 0 {
avgSeconds := metric.Histogram.GetSampleSum() / float64(metric.Histogram.GetSamp
return  time.Duration(avgSeconds * float64(time.Second))
```

```
}
return 0
}
```

8.7.2 자동 튜닝

```
type autoTuner struct {
monitor *performanceMonitor
// 튜닝 파라미터
targetThroughput float64
targetLatency time.Duration
// 현재 설정
currentBatchSize int
currentTimeout time.Duration
currentWorkerCount int
// 튜닝 히스토리
tuningHistory []tuningRecord
// 학습 모델
model *regressionModel
}

type tuningRecord struct {
timestamp time.Time
```

```go
batchSize int
timeout time.Duration
workerCount int
throughput float64
latency time.Duration
}

func (at *autoTuner) Start() {
ticker := time.NewTicker(time.Minute * 5)
go func() {
for range ticker.C {
at.tune()
}
}()
}

func (at *autoTuner) tune() {
// 현재 성능 측정
currentThroughput := at.monitor.GetThroughput()
currentLatency := at.monitor.GetAverageLatency()
// 기록 저장
record := tuningRecord{
timestamp: time.Now(),
```

```
batchSize: at.currentBatchSize,
timeout: at.currentTimeout,
workerCount: at.currentWorkerCount,
throughput: currentThroughput,
latency: currentLatency,
}
at.tuningHistory = append(at.tuningHistory, record)
// 모델 업데이트
at.model.Train(at.tuningHistory)
// 최적 파라미터 예측
optimalParams := at.model.Predict(at.targetThroughput, at.targetLatency)
// 파라미터 적용
at.applyParameters(optimalParams)
}
```

8.8 벤치마킹

8.8.1 성능 테스트 프레임워크

```
type benchmarkSuite struct {
scenarios []benchmarkScenario
results []benchmarkResult
}
```

```go
type benchmarkScenario struct {
name  string
nodeCount  int
faultCount  int
workload workloadGenerator
duration time.Duration
}

type benchmarkResult struct {
scenario  string
throughput  float64
latency time.Duration
cpuUsage  float64
memUsage  int64
networkIO  int64
}

func (bs *benchmarkSuite) Run() {
for  _, scenario := range  bs.scenarios {
result := bs.runScenario(scenario)
bs.results = append(bs.results, result)
}
```

```
}

func (bs *benchmarkSuite) runScenario(scenario benchmarkScenario) benchmarkResul

// 테스트 환경 설정

testEnv := bs.setupTestEnvironment(scenario)

defer  testEnv.Cleanup()

// 모니터링 시작

monitor := newPerformanceMonitor()

monitor.Start()

// 워크로드 실행

scenario.workload.Generate(scenario.duration)

// 결과 수집

return benchmarkResult{

scenario: scenario.name,

throughput: monitor.GetThroughput(),

latency: monitor.GetAverageLatency(),

cpuUsage: monitor.GetCPUUsage(),

memUsage: monitor.GetMemoryUsage(),

networkIO: monitor.GetNetworkIO(),

}

}
```

이러한 성능 최적화 기법들을 통해 HotStuff는 대규모 분산 환경에서도 높은 처리량과 낮은 지연시간을 달성할 수 있으며, 다양한 네트워크 조건과 워크로드에 적응할 수 있습니다.